Shadow-AI Radar: Project Summary

A local, dependency-free tool that finds unsanctioned AI usage and sensitive-data leakage in a company's web-proxy logs. Status as of 2026-10-28: engineering complete for local use; validation on real corporate data not yet done.

---

1. The problem

Employees paste source code, customer data and credentials into consumer AI tools (ChatGPT, Gemini and others) that the company never vetted. Most security teams cannot see it. Shadow-AI Radar reads egress-proxy logs and answers four questions: **who uses which AI tools, which of those are unapproved, what sensitive data went into them, and what should the security team do next?**

Intended users: a security / GRC team at a company, working alongside an HR, Legal and privacy function.

2. How it works

```
proxy logs (Squid / CSV / JSONL)
   → convert    common schema; URLs with spaces handled; error lines skipped
   → ingest     parallel, incremental, crash-safe; redact-before-store; opt. pseudonymization
   → enrich     AI-tool catalog + unknown-AI discovery; secret/PII scan; sensitivity classifier
   → store      SQLite: events, user-day, service, department, week
   → detect     5 alert types, correlated and de-duplicated
   → act        dashboard · CLI triage · blocklist · SIEM JSONL · CSV · notifications
```

Alerts: **SECRET_LEAK** (keys, tokens, passwords, private keys), **PII_LEAK** (SSN, Luhn-valid cards), **EXFIL_VOLUME** (unusually large uploads vs. department peers), **SHADOW_ADOPTION** (an unvetted tool spreading across users), **OFF_HOURS** (activity at hours unusual for that user).

3. The four disciplines

Discipline	What was built
Data engineering	Bronze → silver → gold pipeline on SQLite; process-pool parallelism; byte-offset checkpoints so re-runs read only new data; one transaction per 5,000-line chunk; data-quality rejects (malformed, duplicate, bad timestamps); retention; log adapters
AI engineering	Prompt-sensitivity classifier (public / internal / confidential / source code) with labelling → training → evaluation loop (active learning, held-out sets, per-class precision/recall); optional AI triage summaries with an offline fallback; classifier down-weighted until trained on real data
Data analysis	Robust median/MAD volume anomalies vs. department peers; per-user baselines for off-hours; adoption tracking; risk scoring (tool policy × data sensitivity); week-over-week trend normalised per active day

Discipline	What was built
Cybersecurity	Secret/PII detection (patterns, Luhn, entropy, context rules); redact-before-store; ReDoS-hardened parsing; tamper-evident audit log; authenticated dashboard; encrypted backups; threat model

4. By the numbers

Metric	Value
Code	~2,100 lines across 21 modules; ~1,250 lines of tests; ~500 lines of document tooling; **zero third-party dependencies** (Python 3.10+)
Tests	**87 passing**; full run ≈ 90 s; no leftover temp files
Speed (8 cores)	≈ 16–21k events/s; 255k events ingested in ≈ 12–15 s; re-run with no new data 0.06 s
Interface	23 CLI commands, a localhost dashboard, JSON API
Docs	README, ROADMAP, and `docs/`: PRIVACY, SECURITY, COMPLIANCE, PILOT, REAL_DATA_TEST, and this summary (Markdown, PDF, Word)

5. Capability inventory

- **Operate:** policy-as-config, alert lifecycle (open / ack / resolved / false positive, notes, auto-reopen on recurrence), dashboard with analyst/viewer roles, watch mode, systemd/cron generation (generates only, installs nothing), desktop notifications, analyst training mode (8 hidden cases with an answer key).
- **Integrate:** Squid and generic CSV converters, HR/IdP join (managers on alerts), proxy blocklist, SIEM-friendly JSONL, audit CSV with hashes.
- **Protect (the tool itself):** named users, constant-time token checks, lockout, Host/Origin checks, CSP and other headers, owner-only files (`umask 077`), hash-chained audit log with off-box anchors, AES-256 encrypted backups verified on restore, log-injection and CSV-formula-injection guards.
- **Comply:** keyed pseudonymization of user IDs, data-subject export and erasure, automatic 90-day retention, `radar doctor` posture check, `radar evidence` auditor bundle, GDPR / SOC 2 / ISO 27001 control mapping with an explicit gap list.

6. What testing found (and what was fixed)

Testing was adversarial and used real-world inputs, and it found real defects in the project's own code:

Found by	Defect	Outcome
Hardening review	A crafted ~10 KB prompt hung the card detector for minutes (ReDoS)	Linear matcher, bounded patterns, windowed scanning

Found by	Defect	Outcome
Real Squid formats	URLs containing spaces attributed events to the **wrong user**; error lines became junk events	Parser anchored from both ends; error lines skipped
406 real WildChat prompts	**Every** critical alert (2 of 2) was a false positive: URL slugs flagged as secrets	URLs excluded from the entropy scan; new rules for secrets inside URLs; 0 false alerts afterwards
Same real-data test	Bootstrap classifier scored 62% on real prompts, **worse than guessing "public" (80%)**	Contribution to risk capped at 35% until trained on real labels
Stress loop (6 modules in parallel)	The WildChat fix had reintroduced a ReDoS (6–9 s CPU on 200 KB)	Fixed; replaced a flaky wall-clock test with a CPU-time test calibrated against harmless text, verified to catch the old bug
Same stress run	Tests leaked 150–200 MB each run and filled a RAM-backed `/tmp`	Tests now clean up after themselves (0 MB growth)
Security review of own work	Rejected requests were not audited; data folder permissions not tightened if pre-existing	Both fixed

7. Honest limitations

1. **Never run on real corporate traffic.** Detection quality on real secrets and PII is unproven. The WildChat data is pre-anonymised, so it measured false positives and robustness, not recall.
2. **The classifier is weak.** 80% on a synthetic held-out set (confidential recall 50%); 62% on 40 real English prompts labelled by one person. It is advisory only and not meaningful for non-English text.
3. **Live database is not encrypted** by the tool (Python's standard library has no vetted cipher). Use disk encryption. Backups are encrypted.
4. **Dashboard is plain HTTP on localhost** with token logins. No TLS or SSO, so it is not for network exposure.
5. **Not independently reviewed.** No external penetration test or audit.
6. **Squid's native format has no upload size,** so large-upload alerts are blind on it unless `%>st` is added to the log format.
7. **Compliance is supported, not achieved.** The tool provides controls and evidence; the organisation must supply the lawful basis, employee notice, signed DPIA, access reviews and the rest (see `docs/COMPLIANCE.md`).
8. **Systemd units are validated but have not run as a live service.**
9. One earlier test failure was never captured; it is most likely the timing test that was later fixed, but this was not confirmed directly.

8. What remains (needs people or real data, not more code)

#	Item	Owner
1	Pilot on one department's real proxy logs in shadow mode (`docs/PILOT.md`)	Security
2	Label ≈ 500 real prompts, retrain, reach ≥ 90% accuracy and ≥ 90% confidential recall on a held-out set	Security / data
3	Complete and sign `docs/PRIVACY.md`; establish lawful basis; notify employees (works council where required)	Legal / HR / DPO
4	Encrypt the host disk	IT
5	Independent penetration test	Security
6	Install the generated schedule; store audit anchors off-box	IT / Security

`radar doctor` warns about items 3 to 5, and about a missing audit anchor (part of item 6), until they are done. It does not check whether the schedule is installed.

9. Quick start

```
python3 -m radar run --rebuild                 # demo: synthetic data → out/report.html
python3 -m radar training start                # practice triage with a hidden answer key
RADAR_USERS="alice=analyst:<16+ char token>" \
  python3 -m radar serve                       # dashboard (refuses to start without logins)
python3 -m radar convert squid access.log out/proxy.jsonl --identity hr.csv
python3 -m radar doctor                        # security / privacy posture
python3 -m unittest discover tests             # 87 tests, ≈ 90 s
python3 tools/build_docs.py                    # rebuild this summary as PDF and Word
```

10. Repository map

```
radar/    21 modules: pipeline, detectors, catalog, model, analytics, adapters, serve, audit,
          privacy, compliance, backup, export, notify, schedule, training, labeling, triage,
          report, generate, corpus, __main__
tests/    7 test files + _cleanup.py        eval/    held-out classifier sets (synthetic)
config/   policy.json                       bench/   throughput benchmark
docs/     PRIVACY · SECURITY · COMPLIANCE · PILOT · REAL_DATA_TEST · PROJECT_SUMMARY
tools/    build_docs · md2html · md2docx · verify_docx   (Markdown → PDF / Word)
```

11. Build history

Phase	Delivered
1	Detection core: ingest → enrich → detect → report, synthetic data with planted incidents
2	Operable product: policy config, alert lifecycle, exports, dashboard, watch, triage, user risk
3 / 3.5	Log adapters, measured classifier, HR join, retention, dashboard auth, labelling tool, pilot runbook

Phase	Delivered
4 / 4.5	Security hardening, audit log, encrypted backups, pseudonymization, DSR, doctor, evidence
4.7	Real-data test on WildChat: found and fixed false-alert and Squid bugs
4.8	Stress testing: found and fixed a ReDoS regression and a test-suite leak
5	Trends, CSV export, scheduling, notifications, off-hours detector, analyst training
5.1	Document tooling: Markdown → PDF / Word with a content-exact verification step

12. What makes it distinctive

- It spans **data engineering, AI engineering, data analysis and cybersecurity** in one coherent product aimed at a current corporate risk (shadow AI), not a toy dataset.
- It is **privacy-first by construction:** secrets and PII are redacted before storage, identities can be pseudonymized, and subject-rights and retention are built in.
- It **reports its own weaknesses:** measured numbers, a posture checker that warns instead of claiming, and a documented list of what it does not do.
- It **runs anywhere** with only the Python standard library.